Exercise - Simulation flow

Overview

Verification is a critical aspect of IC design. A typical verification flow involves comparing a circuit implementation—whether written in SystemVerilog RTL or as a gate-level netlist—against a golden model that represents the expected behavior. The verification process applies stimuli vectors (input signals) to the circuit under test, collects the outputs, and compares them against the expected results from the golden model. Any mismatches are flagged in a report.

Although writing testbenches was covered in the VLSI 1 course and is not the primary focus of this lecture, we will extensively use existing verification setups. In this exercise, we will start with a simple 8-bit adder, familiarize ourselves with the simulation flow, and then move to a more complex design: the example SoC, called Croc. Both designs include testbenches to verify basic functionality. We will modify these testbenches as needed to accommodate the additional functionality introduced during the course.

Initially, the designs will be described at the RTL level in SystemVerilog. We will use Verilator, a leading open-source tool for high-performance RTL simulation.

In the second part of the exercise, we will explore gate-level simulations, which include back-annotated delays to reflect the parasitic effects of physical design. However, due to the lack of a robust open-source setup for gate-level simulations, we will use QuestaSim (a commercial tool by Mentor/Siemens) for this part of the exercise.

Goals of this exercise

- Gain hands-on experience with RTL simulation using Verilator.
- Learn the testbench template that will be used throughout the exercise.
- Understand post-layout simulation with back-annotation, including the additional steps and constraints involved.
- Develop skills in debugging simulation errors by tracing them to their root causes.
- Re-run existing testbenches to verify that modifications to the SoC do not introduce new bugs.

About the style

We will use a number of different styles to identify different types of actions as shown below:

Student Task

Parts of the text that have a gray background, like the current paragraph, indicate steps required to complete the exercise.

Actions that require you to select a specific menu will be shown like the following: `menu → sub-menu → sub-sub-menu`

Whenever there is an option or a tab that can be found in the current view/menu we will use a `BUTTON` to indicate such an option.

Throughout the exercise you will be asked to enter certain commands using the command-line.

```
sh> command to be entered on the Linux command line
```

Note: *there are many reasons for using a command-line. Some functionality can not not be accessed through GUI commands, and in some cases, using the command-line will be much faster. Most importantly, things you enter on the command-line can be converted into a script and executed repeatedly.*

Preparation

Student Task 1

For ETH users:

- Setup the working directory for this exercise by calling our install script and navigating to the newly created directory. **Important**: make sure you install your exercises on the local hard disks rather than in the accounts home directories (which have a limited disk quota). For this sake, create a directory with your username on the local hard disk (e.g. `/scratch/vlsi2_XXfs26/`), and install the exercise files there. Check out the dedicated Exercise setup page for more information.

```
sh > mkdir -p /scratch/$USER
sh > cd /scratch/$USER
sh > vlsi2 ex01
sh > cd ex01
```

Alternatively, you can directly call the install script for each exercise which you can find under `/home/vlsi2/ex01/install.sh`.

For non-ETH users:

Note: The exercises have been recently updated for the Spring semester 2026. It is possible that some exercise branches on Github require some additional time to be updated accordingly. Please reach out to a maintainer in case of issues.

- Retrieve the files from this exercise from Github:

```
sh > git clone https://github.com/pulp-platform/croc.git
sh > cd croc
sh > git checkout ex01
```

Functional Verification with Testbenches

8-bit Integer Adder

We begin with a simple 8-bit adder to refresh our understanding of simulation and testbenches. Adders are fundamental building blocks of larger circuit components, such as ALUs. In this exercise, you will work with a pre-designed 8-bit word-level full adder to familiarize yourself with the functional simulation flow in Verilator.

File-Based Testbenches

Once a circuit is described in SystemVerilog, it must be functionally verified against the specification using an HDL simulator such as Verilator. This requires a comprehensive verification environment, known as a testbench, which performs the following tasks:

- Generate and apply test stimuli to the Design Under Test (DUT).
- Capture and record the DUT's actual responses.
- Compare the DUT's output against expected responses loaded from a file.

A common approach to testing is a file-based testbench, as illustrated in Figure 2. The advantage of this method is that stimuli and expected responses can be modified without recompiling the testbench, making the verification process more flexible and efficient.

In this part of the exercise, we will use the following Python scripts: `gen_stimuli.py` – Randomly generates input stimuli for the testbench. `gen_exp_response.py` – Serves as the golden model, generating expected responses. These scripts help us to efficiently validate our circuit using a structured, file-based verification approach.

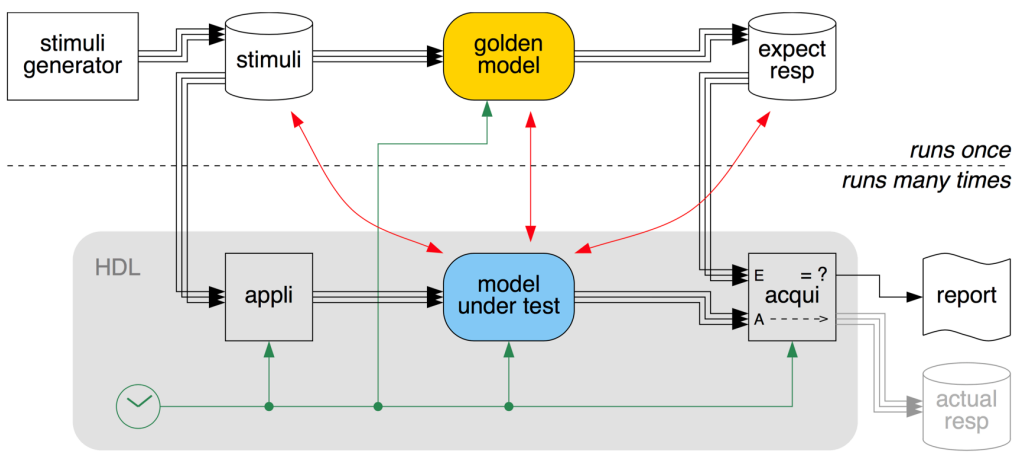

Figure 2: File-based testbench

The file-based testbench written in SystemVerilog typically consists of three main processes:

- Clock and Reset Generation
  - Generates a periodic clock until the end-of-computation (`eoc`) signal is asserted.
  - Generates a low-active reset (`reset_n`) signal at the beginning of the test to initialize the DUT.
  - The clock period and duty cycle are usually configurable through parameters at the start of the testbench.

- Stimuli Application
  - Reads the pre-generated input stimuli from a file into an array.
  - Applies each stimulus from the array on every clock cycle until all stimuli have been processed.
  - Ends the simulation by asserting the `eoc` signal when all stimuli have been applied.
- Response Acquisition and Verification
  - Reads the expected responses from a file.
  - Compares the DUT's actual outputs against the expected responses.
  - If a mismatch is detected, error information is displayed for debugging.

Student Task 1

- Navigate to the `adder8` directory and open the testbench file `sourcecode/tb_adder8.sv` using your preferred text editor (e.g. Vim, Emacs, Sublime Text (`sublime_text`, VSCode (`code`), etc.).
- How many cycles will the DUT need for resetting?

Next, we will walk through the steps required to simulate the design in Verilator and debug the circuit using waveform analysis.

RTL Simulation with Verilator

Simulating a design with Verilator happens in three stages: the design is first translated to either SystemC or C++, it is then compiled to an executable binary on the host PC, and finally, the binary is run. This process enables cycle-accurate hardware execution on a host PC, which typically has a different hardware architecture than the design itself.

To compile the design into an executable binary, we need to provide some information to Verilator, e.g., the paths and names of the design files, and the name of the top-level design. For this simple design, we can specify all source files in a single command. However, for larger designs, we typically use a file list containing all required source files.

Student Task 2

- First, open a bash session with `oseda`. This will add the required programs in your `$PATH`.

```
sh (ex01) > oseda -2025.12 bash
```

- Complete the following command by replacing the `/* */` placeholders with the appropriate filenames and module names to compile the design into a binary:

```
sh (ex01)   > cd adder8/
sh (adder8) > verilator /*design and testbench file names*/ --top /*top module name*/ --binary -Wno-fatal --trace --trace-structs
```

The `--trace --trace-structs` options enable waveform generation while preserving hierarchical structures, making signal visualization easier for debugging. We will discuss and use these trace files in a later section of the exercise.

After compilation, Verilator creates an `obj_dir` folder in the working directory. This folder contains the compiled binary along with other necessary files used by the tool.

Now, we can proceed with running the simulation by executing the compiled binary.

Student Task 3

- Run the simulation by executing the following command:

```
sh (adder8) > ./obj_dir/Vtb_adder8
```

- Does the simulation complete successfully? Are there any reported mismatches?

GTKWave

The messages reported in the terminal only indicate whether the design passed or failed by comparing the DUT's outputs against the golden model. This provides a quick verification check but does not help identify the root cause of failures.

To debug issues in our Adder8 design, we need to observe the circuit's behavior during simulation. Since Verilator does not have a built-in waveform viewer, we must export the waveform data to a Value Change Dump (VCD) file during simulation and open it using an external tool. In this exercise, we will use GTKWave to visualize the signal traces.

As mentioned earlier, the `--trace --trace-structs` option tells the tool to record the wave files. Besides, we need to tell the tool the time and signal names we want to record, as well as the location where we want to store the file. These settings are configured within the testbench. We have already included the necessary commands in the Stimuli Application section of the testbench, so you won't need to modify them.

Student Task 4

- Before opening the waveform, take a moment to re-examine the printed messages from the previous task. Identifying patterns in the failed input vectors can help narrow down the debugging scope. Can you find any common characteristics among the failed cases?
- The VCD file `adder8.vcd` was generated in the previous step. We can now open this file with GTKWave to visualize how the design is working during the simulation.

```
sh (adder8) > gtkwave adder8.vcd
```

- After running the command, the GUI shown in Figure 3 should open. On the left column, you will see the design hierarchy with `tb_adder8`. To import all signals from the testbench and DUT, you can either right-click on the `tb_adder8` and select `Recurse Import -> Insert`, or select the `tb_adder8` and click the `Insert` in the bottom left. These steps will load all available signals, allowing you to analyze how the design behaves during simulation.
- Now, we can click the `Zoom Fit` in the top toolbar to observe all the transactions. Share your thoughts with a TA and try to fix the bug.

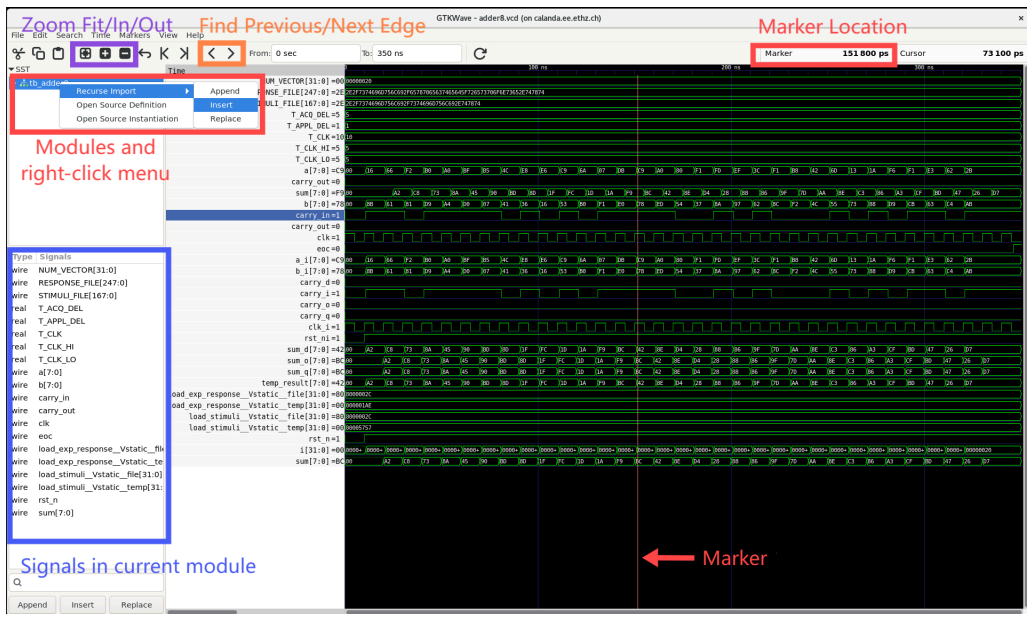

Figure 3: GTKWave GUI

Croc SoC

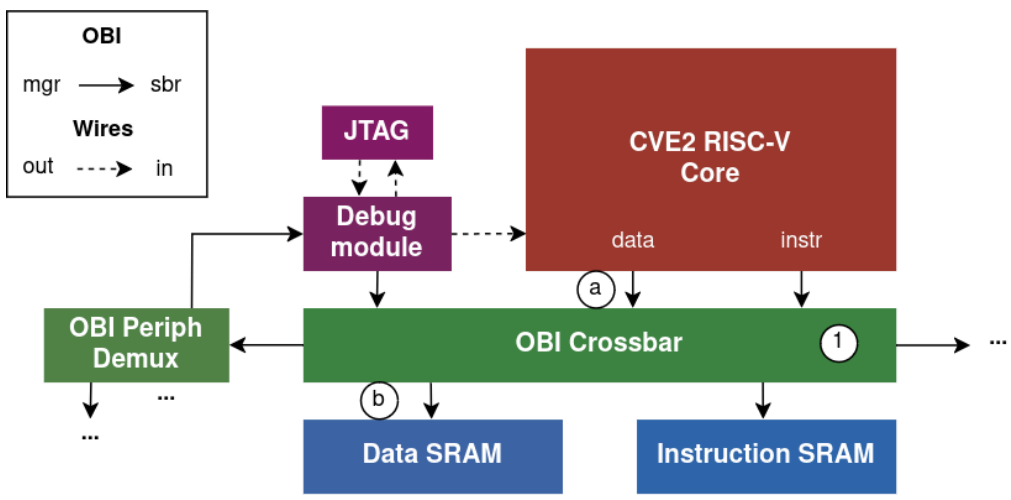

Figure 4: Simplified Croc diagram

We have now refreshed our knowledge of simulation and learned the basic flow for compiling, simulating, and debugging using the open-source tools Verilator and GTKWave. With this foundation, we will now shift to a more complex design: the Croc SoC, which will be the primary focus of this course.

This exercise will not dive deep into the microarchitecture of the Croc SoC—that will be covered in exercise 3. For now, here's a brief introduction:

- Croc is a 32-bit microcontroller system based on the RISC-V CVE2 (previously, Ibex) core.
- The system includes two 2KiB SRAM banks.
- This Harvard-style architecture of the CVE2 core exploits this by simultaneously accessing data and instructions, improving performance.
- The Croc SoC supports on-chip debugging and programming via a standard JTAG interface, enabled by the RISC-V debug module, as illustrated in Figure 4.

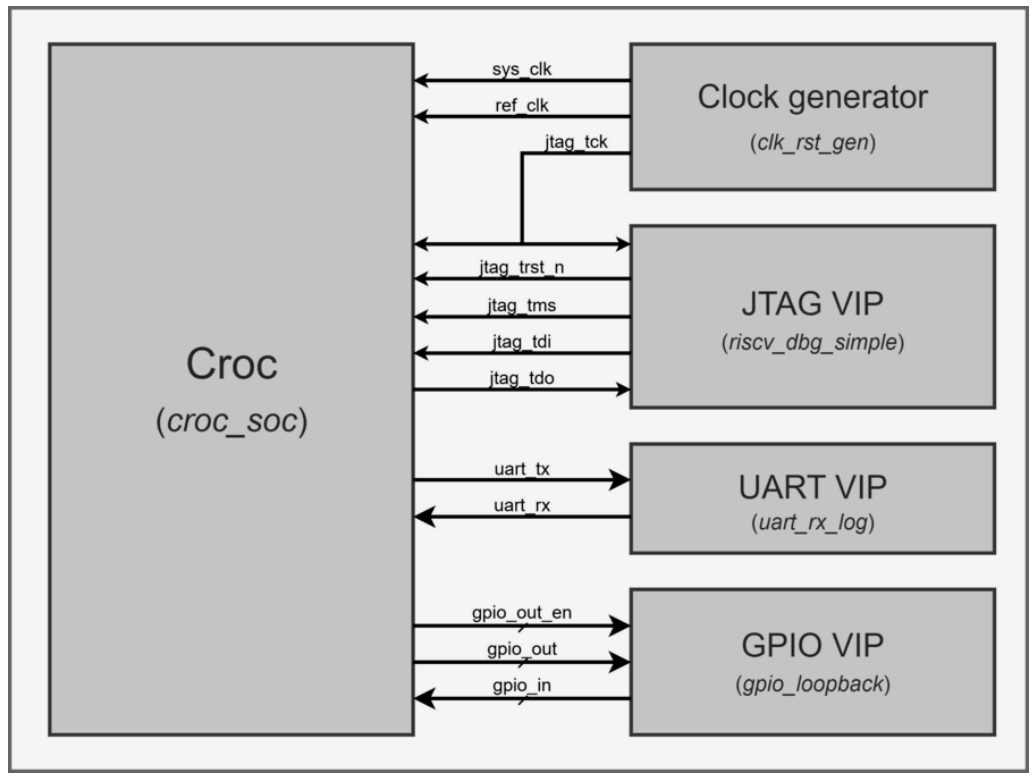

Figure 5: Croc testbench overview

Although the testbench for the Croc SoC follows the same fundamental principles as the Adder8 design, it is more complex because the SoC can execute different programs rather than just processing static input vectors.

Figure 6 shows the simulation flow for the Croc SoC. In a nutshell, the process is divided into the following key steps:

- **Software Compilation**
  - The simulation starts with C-based tests, which are compiled into RISC-V assembly code and RISC-V executable binaries.
  - The input stimuli and expected responses are typically embedded within these C-based tests.
- **RTL Simulation**
  - Similar to the Adder8 example, the RTL source code is compiled into an executable model for the host PC.
  - The simulation is then launched, executing the compiled binary.
- **Croc Testbench Sequence**
  - During simulation, the JTAG loader in the testbench reads the RISC-V executable binary and loads it into the SRAM banks. Once loaded, the core is woken up and begins executing instructions from the entry point.
  - The core writes to a memory-mapped register, asserting the eoc (end-of-computation) signal upon completion.
  - The JTAG module continuously polls this register until the eoc signal is detected, at which point the simulation ends.

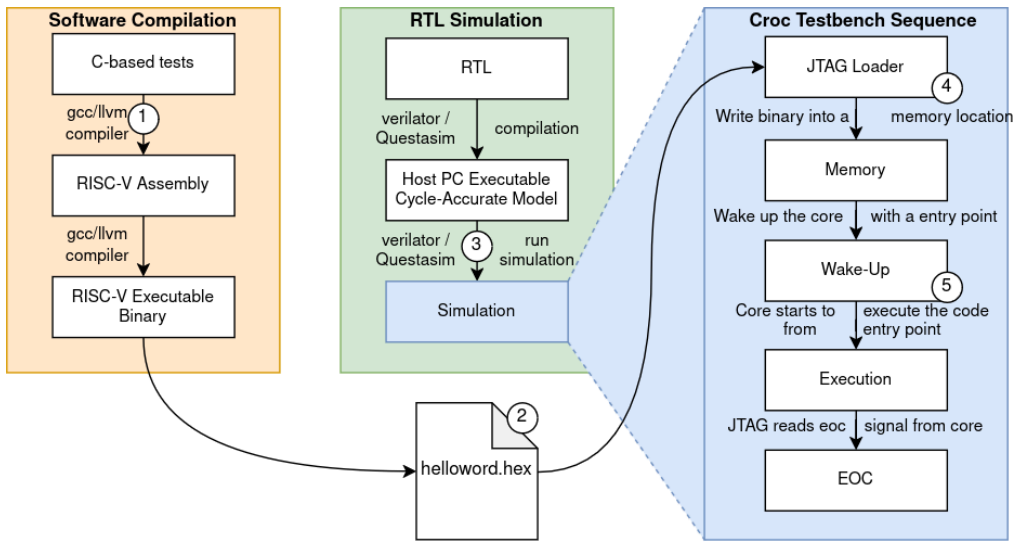

Figure 6: Croc testing flow

Since the compilation process is beyond the scope of this exercise, we have included the relevant pre-compiled binaries in the files of this exercise. The binaries are located in the `sw/bin` directory. During simulation, the testbench will load the specified binary into the SoC's SRAM via the system's JTAG interface.

We are now almost ready to compile and simulate the SoC. However, unlike the Adder8 design, our SoC consists of multiple RTL files, making it impractical to manually specify each file in the compilation command.

To simplify this process, we use a file list containing all RTL source files required for the simulation and the necessary macro definitions used in those files.

Student Task 5

- Navigate to the `verilator` directory

```
sh (ex01) > cd verilator/
```

- Open the `croc.f` file. How many files are being compiled for the simulation?
- Run the following command to compile the design:

```
sh (verilator) > verilator --binary -j 0 -Wno-fatal --trace --trace-structs --top tb_croc_soc -f croc.f
```

Note: the first-time compilation may take a few minutes. If no RTL changes are made, you do not need to recompile the design for subsequent runs.

- Once compiled, execute the binary to start the simulation:

```
sh (verilator) > ./obj_dir/Vtb_croc_soc
```

- Examine the test output. Did the simulation produce the expected results?

Now that we have completed the SoC simulation, we can use the waveform traces to examine key events in the execution sequence, including program loading and the assertion of the end-of-computation signal.

As before, the simulation has recorded a VCD file for waveform analysis. However, unlike the small Adder8 design, the SoC simulation generates a much larger VCD file, potentially occupying hundreds of MiB of storage.

- When extending the SoC or running longer simulations, excessive VCD file sizes can quickly consume disk space.
- Verilator provides options to trade off storage size for simulation visibility. `--trace-depth 1` Limits tracing to top-level signals to reduce file size. Adjusting the tracing scope directly in the testbench can also help optimize storage.

For today's exercise, you can keep the default settings, which record all signals, to ensure full visibility during debugging.

Student Task 6

- The VCD file has be generated in the `verilator` directory. How large is this file?

- Open the waveform file using GTKWave. Since the SoC is complex, avoid importing all signals at once. Focus on the JTAG signals and memory interface, which are essential for the boot sequence. Add the five JTAG interface signals to the waveform view at the croc_soc level: `tb_croc_soc/i_croc_soc`, and the main memory interfaces at `tb_croc_soc/i_croc_soc/i_croc/gen_sram_bank[0]`.
- Before the JTAG module loads the executable binary, a simple memory write & read test is performed. Identify the time and data of memory write and read operations. Compare these with the test log output. Do the timestamps match? If not, why?
- After the simple memory test, JTAG writes the binary code to the starting address. What data is written to address `0x1000_0008`? Open the `sw/bin/helloworld.dump` file, which contains the assembly code for the test we execute. Identify the assembly instruction fetched at this moment.
- The `eoc` signal is set as part of a memory-mapped register (named `CORESTATUS`) inside the `tb_croc_soc/i_croc_soc/i_croc/i_soc_ctrl` module. The interface between the SoC and the `soc_ctrl` module is implemented by the `soc_ctrl_obi_req` and `soc_ctrl_obi_rsp` signals. Can you determine the full memory address of the `CORESTATUS` register containing the `eoc` signal?

Hint: You may check the input requests to the main xbar at the cycle `eoc` signal is toggled.

Now that you have learned how to simulate an RTL design using Verilator and debug it effectively, you are well-equipped to test your circuit whenever you make changes.

To further improve your verification skills, you may write your own test cases to cover additional scenarios and ensure your design behaves correctly under different conditions. This will help you identify and fix potential issues early in the design process.

Let's move on to the second part of the exercise on post-layout simulation.

Post-Layout Simulation

Published with permission from Siemens. Siemens QuestaSim is available through EDA Higher Education Software Program (https://www.sw.siemens.com/en-US/academic/educators/eda-higher-education-software/)

In this second part, we will move forward in the design chain to learn how to use the simulator for post-layout simulation – i.e., the simulation of a gate-level netlist after the physical design stage (place and route) with additional information about propagation delays.

While there are open-source solutions that support such simulations, we were unable to establish a flow until now, and will use Siemens QuestaSim for this part of the exercise.

We have prepared an introduction about the tool in Intro to Questasim. You can refer to this introduction on the commands you need for the exercise. The ASIC design flow is described in detail in the textbook1. The figure below gives a graphical overview of the subject of this exercise.

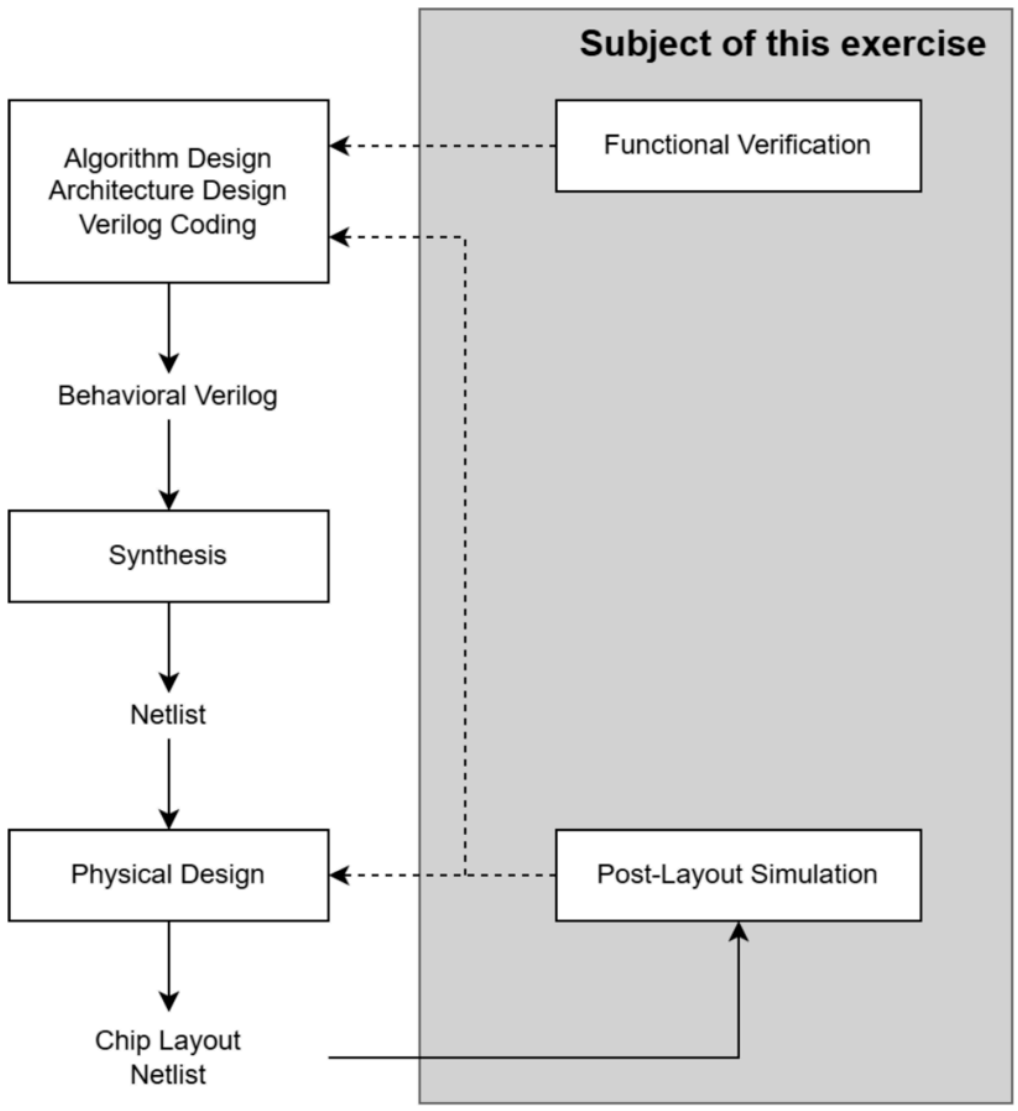

Figure 7: Content of the ASIC design flow with the subject of this exercise highlighted.

Post-Layout Simulation with Siemens QuestaSim

After verifying the RTL model, the next step in the design flow is to synthesize a gate-level netlist and finally generate a physical layout of the design.

At the end of the design process, another round of functional verification is required to ensure that no errors have been introduced by synthesis, placement, or routing. Instead of using an RTL model, the Device Under Test (DUT) in this stage will be a gate-level netlist extracted from the final chip layout. You will learn how to generate a netlist from a design in OpenROAD in a later exercise. For this exercise, we provide you an extracted post-layout netlist of the Croc SoC in: `out/croc_pls.v`.

To avoid replicating code, we will reuse the same testbench as before, named `tb_croc_soc` (see `rtl/test/tb_croc_soc.sv`). However, since the post-layout netlist also includes IO pads, some minor modifications in the instantiation of Croc are required to correctly connect the DUT and the other components of the testbench, mostly related to the handling of bi-directional signals (e.g. GPIO pads). In Croc, the instantiation of the DUT is controlled by custom macros, allowing the maximum reuse of other testbench components. Take a look at the testbench to get a feeling of how this is implemented.

Student Task 7

Let's start by having a look at the provided compilation scripts for post-layout simlulation of Croc. Open the `vsim/compile_netlist.tcl` script in a text editor and make sure that the `TARGET_POSTLAYOUT` macro is defined when compiling the testbench. This ensures that the correct instantiation of Croc is used in the testbench (see `rtl/test/tb_croc_soc.sv`).

This script is roughly the equivalent of the "file list" we used to compile the design with Verilator. The main difference is that it is not just a list of the Croc source files needed to run the simulation, but rather a script that runs the commands to compile each source file with QuestaSim. The main command to compile Verilog/SystemVerilog files in QuestaSim is `vlog` (Intro to Questasim).

Since we are simulating a gate-level netlist that includes technology-specific cells, we must also compile the technology libraries. They have already been included inside the `compile_netlist.tcl`. Take a quick look at the `compile_rtl.tcl` and `compile_netlist.tcl` scripts, to see which files are required to compile RTL and gate-level netlist designs. Which libraries are we using for Croc?

To guide you through the post-layout simulation process, we have prepared a template script for running the simulation. Your following tasks are to:

1. Complete the missing commands inside the template file.

2. Execute the script in the shell.
3. Refer to Intro to Questasim for more details on QuestaSim commands.

Student Task 8

- First, copy in the required Verilog models used to generate the provided netlist by running the `init_tech.sh` script in the root directory of this exercise:

```
sh (ex01) > ./init_tech.sh
```

- Then, navigate to the `vsim` directory and start Siemens QuestaSim.

```
sh (ex01) > cd vsim
sh (vsim) > questa-2019.3 vsim
```

In this exercise, we use QuestaSim 2019.3 for post-layout simulation. Although newer versions exist, we stick to 2019.3 to ensure compatibility with the library cells' Verilog behavioral models, which have been tested in this version.

- Create a new design library named `work` using the `vlib` command.
- Compile the netlist and the testbench by sourcing the `compile_netlist.tcl` script.
- The compiled testbench (`tb_croc_soc`) needs to be optimized for simulation. Optimize the design using the `vopt` command.
- Use the `vsim` command to launch the optimized testbench. You also need to include the technology libraries and suppress an irrelevant warning caused by the missing timeunit/timeprecision in the SRAM model, which can be safely suppressed. To suppress the error, add the following option to your command: `-suppress vsim-3009`.
- In QuestaSim, open the *sim* window and select the DUT instance `i_croc_soc`, which is instantiated inside the testbench. The *Objects* window will display all signals in the selected module. To monitor all input, output, and inout signals at the DUT, add them to the *Wave* window by:
  - Dragging them from the *Objects* window to the *Wave* window.
  - Or, using the shortcut `Ctrl+W` .
- You can run the simulation for a predetermined time by typing:

  ```
  vsim> run Xns
  ```

  Replace **X** with a time long enough for the JTAG interface to complete a simple Write/Read test, as determined from the first part of this exercise.
- Run the simulation. Do you observe any error messages in the simulation log? If so, analyze the errors and try to determine their causes.

Note: gate-level netlist simulation may take a long time depending on the level of optimizations you turned on (be mindful of the common `+acc` option!).

Post-Layout Simulation with Timing Annotation

So far, we have only simulated the design using the post-layout netlist and real technology cells. The timing information has not yet been included in our simulation flow.

An additional timing file can be exported when generating the netlist after the design has been completed. This file describes the delays across the logic gates in the netlist based on the information available after place and route, and is stored in the Standard Delay Format (SDF).

In OpenROAD, we can use the OpenSTA engine with `write_sdf OUTPUT_PATH -corner CORNER -include_typ` command to generate the SDF file at a specific corner. More details will be provided in future lectures and exercises. We have prepared an SDF file `out/croc.sdf` for you today.

This file describes:

- Cell delays: The real delays introduced by logic gates.
- Wire delays: The additional delays due to interconnect routing.

- Accurate setup/hold timing checks: Derived from extracted RC information based on the library database.

At the beginning of the SDF file, the Process, Voltage, and Temperature (PVT) conditions for each process corner are specified as follows:

```
(VOLTAGE 1.200:1.200:1.200)
(PROCESS "1.000:1.000:1.000")
(TEMPERATURE 25.000:25.000:25.000)
```

Here, the three columns represent the *minimum*, *typical*, and *maximum* conditions, respectively. Due to the limitation of the OpenSTA engine used by OpenROAD, only one PVT corner is supported for SDF generation. Even when using a single corner, OpenSTA generates different values for the upper bound in maximum, the lower bound in minimum, and the average in typical entries. This is not the case for most tools; they usually output only one delay value per PVT corner.

The delays listed in SDF files often differ from those in datasheets because SDF timing accounts for load-dependent propagation delay, taking into account the actual fan-out and parasitic effects in the placed-and-routed design. This is why timing verification must be performed using the extracted SDF values, as they reflect the actual behavior of the implemented design.

Let's take a look at the timing information of one register cell from the SDF file:

```
(CELL
 (CELLTYPE "DFFRQX3")
 (INSTANCE i_croc_soc\/i_croc\/i_dmi_jtag\/data_q_11__reg)
 (DELAY
  (ABSOLUTE
   (IOPATH CK Q (0.315:0.320:0.325) (0.274:0.278:0.283))
   (IOPATH RB Q () (0.000:0.000:0.000))
   (COND CK == 1'b0
    (IOPATH RB Q () (0.000:0.000:0.000)))
   (COND CK == 1'b1 && D == 1'b0
    (IOPATH RB Q () (0.000:0.000:0.000)))
   (COND CK == 1'b1 && D == 1'b1
    (IOPATH RB Q () (0.000:0.000:0.000)))
  )
 )
 (TIMINGCHECK
   (REMOVAL (posedge RB) (posedge CK) (-0.106:-0.108:-0.109))
   (REMOVAL (COND COND_D == 1'b1 (posedge RB)) (COND COND_D == 1'b1 (posedge CK)) (-0.106:-0.108:-0.109))
   (RECOVERY (posedge RB) (posedge CK) (0.149:0.151:0.154))
   (RECOVERY (COND COND_D == 1'b1 (posedge RB)) (COND COND_D == 1'b1 (posedge CK)) (0.149:0.151:0.154))
   (SETUP (posedge D) (posedge CK) (0.103:0.104:0.106))
   (SETUP (negedge D) (posedge CK) (0.062:0.064:0.065))
   (SETUP (COND COND_RB == 1'b1 (posedge D)) (COND COND_RB == 1'b1 (posedge CK)) (0.103:0.104:0.106))
   (SETUP (COND COND_RB == 1'b1 (negedge D)) (COND COND_RB == 1'b1 (posedge CK)) (0.062:0.064:0.065))
   (HOLD (posedge D) (posedge CK) (-0.051:-0.052:-0.053))
   (HOLD (negedge D) (posedge CK) (0.020:0.019:0.018))
   (HOLD (COND COND_RB == 1'b1 (posedge D)) (COND COND_RB == 1'b1 (posedge CK)) (-0.051:-0.052:-0.053))
   (HOLD (COND COND_RB == 1'b1 (negedge D)) (COND COND_RB == 1'b1 (posedge CK)) (0.020:0.019:0.018))
   (WIDTH (posedge CK) (0.138:0.140:0.141))
   (WIDTH (negedge CK) (0.196:0.198:0.199))
   (WIDTH (negedge RB) (0.322:0.332:0.343))
  )
)
```

The `DELAY` entry specifies the propagation delays from the `CLK` or `RESET_B` to the outputs. For some `DELAY` path, the SDF file may include two entries, representing the rising and falling edge, respectively. The `TIMINGCHECK` entry specifies the setup, hold, recovery, and signal-width time for the specified path.

In this example, the propagation delay from the positive edge of `CLK` to the positive edge of `Q` is 0.320 ns for the typical delay. This information replaces the default delays specified in the Verilog model of the library cells.

You will learn more about timing in future lectures and exercises. Today, we provide only an abstract overview of an SDF file's contents.

Due to the work in progress status of the technology we are using for this year's class, the SDF-annotated post-layout simulation currently has issues. You may need to lower the system and JTAG clock speeds significantly for the simulation to pass (ClkSys~500ns, ClkJtag~2000ns), and adjust the UART baudrate accordingly in the testbench (see `rtl/test/tb_croc_pkg.sv`). However, we encourage you to go through the steps of task 9 and make sure that you understand and are already able to start a simulation of the Croc netlist with

SDF annotations.

Student Task 9

- What is the setup time for a positive edge on port D at the rising clock in the minimum delay case for the cell listed above?
- To enable the time-annotated modeling for the library cells, we need to remove the define `+define+FUNCTIONAL` to disable the functional-only model. Find and apply this in the `compile_netlist.tcl`.
- Now we are going to include the SDF file in the post-layout simulation. If you have recorded your commands in the previous task, open that script file and add the following option to both the optimization and simulation start commands: `-sdftyp tb_croc_soc/i_croc_soc=../out/croc.sdf +sdf_verbose`.

By passing the `-sdftyp` flag to the `vsim` command, we set the simulation to use the average delays based on the typical corner. If we wanted to use the minimum or maximum delays, we would replace them with the `-sdfmin` or `-sdfmax` flags, respectively.

- Due to the lack of some path delay information in the Verilog behavioral models, there is an issue that should soon be fixed by the PDK provider. For now, we need to suppress certain error messages, as QuestaSim will report missing delay paths specified in the SDF file that are not found in the standard cells' Verilog model. Append the following options to the `vsim` command to suppress these warnings: `-suppress vsim-12088 -suppress vsim-12090`.
- Now, you can start the simulation. Before running it, an additional TCL script is needed to disable timing checks on specific cells. These cells include FIFO registers in the Clock Domain Crossing (CDC) paths, which transfer data between asynchronous clock domains. Since the Verilog models of these cells do not account for two asynchronous clock domains, we need to disable their timing checks to prevent false violations. Run the following command in QuestaSim command line:

```
vsim> source ./disable_cdc_check.tcl
```

Alternatively, you can append the script to the `vsim` command with `-do disable_cdc_check.tcl`.

- Notice if you may see lots of `vsim-3819` warning messages on the display. This is caused by the timing violation on the CDC cells. Although we have disabled the checks, a warning message is still printed when the checking condition is met. You can also suppress this warning information, use the same methods above.
- Finally, add all input and output signals of the DUT to the waveform. Run the simulation until the JTAG interface completes the simple Write/Read test. Is there any error this time?
- If no errors are reported and the simple test passes, run the simulation until the UART channel starts printing messages. Zoom into the time when the `uart_tx_o` signal first rises from 0 to 1 (excluding the reset period). Measure the delay between the rising edge of `uart_tx_o` and the previous positive edge of `clk_i`. Can you explain the cause of this delay?

This testbench timing scheme, illustrated in Figure 7, plays a crucial role in transitioning from functional verification to post-layout simulation.

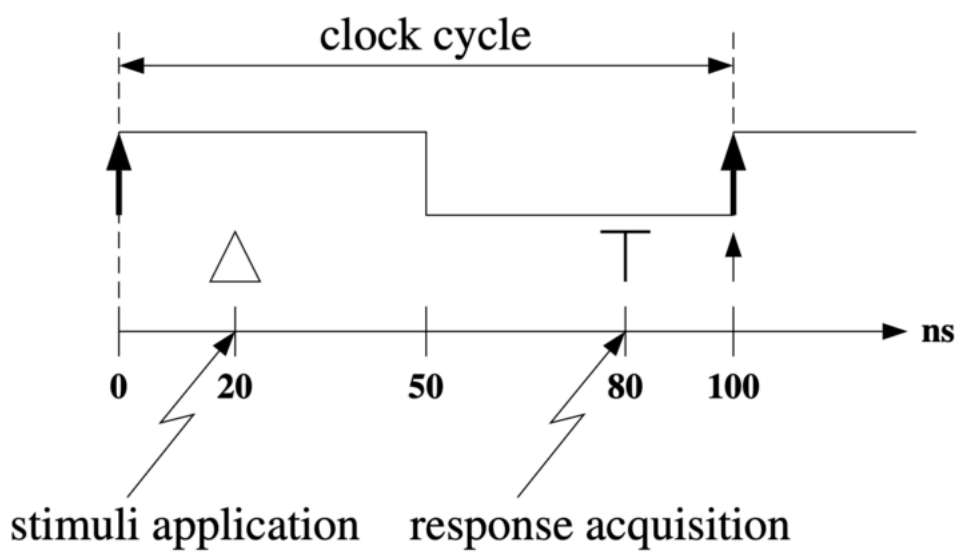

Figure 8: Timing scheme for verification concept.

To model input delay, we apply the stimuli some time before the next rising clock edge. Similarly, we sample the output some time after the rising edge to account for the settling time of the output signals.

It is important to note that the output is not immediately ready before the measured delay time. The response acquisition time ensures that signals are sampled only after they have fully settled, preventing incorrect evaluations.

Student Task 10

- What is the application delay time used in this exercise? Hint: You can find these settings in the testbench.
- What is the response acquisition time?
- What will happen if you change the application delay or acquisition time to a smaller value? Will the simulation still work, and why? Verify your hypothesis by modifying the values, recompiling, and rerunning the simulation.

You are done with this Exercise.

Discuss your experience with assistants and colleagues or continue to explore Verilator and the provided design on your own.

Cleanup

Student Task - Final Cleanup

Only for ETH users:

- If you followed the instruction at this point you have installed the exercise files in a directory on the workstation's local hard disk (e.g. somewhere in `/scratch/$USER`). By default there is no limit to the amount of local disk space that you can use. Please note that the Tardis workstations are shared among several courses and have limited disk space. Therefore, please make sure to cleanup after yourself once the exercise is done. This is particularly important in the case of exercises that generate large files (e.g. long simulation logs, large netlists, and other files generated by the backend tools).

The VLSI pages are part of the open source VLSI design course offered by the Integrated Systems Laboratory (https://iis.ee.ethz.ch) of ETH Zürich (https://www.ethz.ch), by Luca Benini (https://ee.ethz.ch/the-department/people-a-z/person-detail.luca-benini.html) and Frank K. Gürkaynak (https://iis-people.ee.ethz.ch/~kgf/). See full list of contributors. Unless otherwise noted content available under Creative Commons BY (https://creativecommons.org/licenses/by/4.0/).